

Projektdokumentation - MohiBoxer

Ziel

Ziel des Projekts war es, einen trainierten Keras-Agenten für Atari Boxing zu erstellen. Der Agent soll direkt geladen werden können und gegen den eingebauten Atari-Gegner spielbar sein.

Finales Modell:

```
models/mohi_boxer_best.h5
```

Agent

Der finale Agent ist ein Keras-Modell. Er bekommt den aktuellen Spielzustand als vorverarbeitetes Bild und gibt für jede mögliche Aktion einen Wert aus.

Beim Spielen wird keine zufällige Aktion mehr gewählt. Der Agent nimmt einfach die Aktion mit dem höchsten Modellwert:

```
action = argmax(model(observation))
```

Im Code gibt es auch DQN- und DDQN-Agenten. Diese waren der ursprüngliche RL-Ansatz. Für das beste finale Modell war aber Teacher-Student-Training viel schneller und erfolgreicher.

Replay Memory

Beim ursprünglichen DQN/DDQN-Ansatz gibt es Replay Memory.

Replay Memory bedeutet:

- Erfahrungen werden gespeichert.
- Später werden zufällige Beispiele daraus wieder zum Lernen benutzt.
- Dadurch lernt der Agent stabiler.

Im Projekt gibt es:

- normales Replay Memory
- Prioritized Replay Memory
- Replay-Größe: `50,000`

Beim finalen Modell war Replay Memory aber nicht der wichtigste Teil. Das finale Modell wurde hauptsächlich mit einer großen Teacher-Datenmenge trainiert:

- `20,202` gelabelte Episoden
- `8,966,962` Trainingsbeispiele

Diese Datenmenge funktioniert ähnlich wie ein großer Offline-Speicher mit guten Beispielen.

Exploration

Beim ursprünglichen DQN/DDQN wurde Epsilon-Greedy verwendet.

Das bedeutet:

- Am Anfang probiert der Agent viele zufällige Aktionen.

- Später nimmt er immer öfter die beste bekannte Aktion.

Hyperparameter:

- Epsilon Start: `1.0`
- Epsilon Ende: `0.1`

Beim finalen Modell gibt es in der Evaluation keine Exploration mehr. Das Modell spielt deterministisch und wählt immer die Aktion mit dem höchsten Wert.

Reward-Funktion

Es wurde der normale Reward von Atari Boxing verwendet.

Der Agent bekommt Punkte, wenn er Schläge trifft. Es wurde kein eigener zusätzlicher Reward gebaut.

Für das finale Training wurden Sticky Actions aktiviert:

```
repeat_action_probability = 0.25
```

Das bedeutet, dass manchmal die vorherige Aktion wiederholt wird. Dadurch muss der Agent robuster spielen.

Neuronales Netz

Das finale Modell hat:

- Input: `(4, 84, 84)`
- Output: `18`
- Parameter: `1,093,858`

Aufbau:

1. Input: 4 gestapelte graue Atari-Bilder
2. Umordnung in channels-last Format
3. Pixel werden durch `255` geteilt
4. mehrere Convolutional Layers mit Residual Blocks
5. Flatten
6. Dense Layer mit `256` ReLU-Neuronen
7. Output Layer mit `18` Werten

Warum CNN?

Der Agent bekommt Bilder. CNNs sind gut für Bilder, weil sie Positionen, Formen und Bewegungen besser erkennen können als ein normales Dense-Netz.

Observation

Das Atari-Spiel liefert ursprünglich RGB-Bilder.

Vorverarbeitung:

- Bild wird in Graustufen umgewandelt
- Bild wird auf `84x84` verkleinert
- mehrere Frames werden übersprungen

- die letzten 4 Frames werden gestapelt
- Pixel bleiben als `uint8`

Finaler Input:

(4, 84, 84)

Warum 4 Frames?

Ein einzelnes Bild zeigt keine Bewegung. Mit 4 Bildern kann das Modell erkennen, ob sich ein Boxer nach vorne, hinten oder zur Seite bewegt.

Aktionen

Boxing verwendet den vollständigen Atari Action Space:

`Discrete(18)`

Das Modell gibt 18 Werte aus. Jeder Wert steht für eine mögliche Aktion. Die Aktion mit dem höchsten Wert wird ausgeführt.

Training

Das finale Training war Teacher-Student-Training.

Ablauf:

1. Ein starker Teacher-Agent spielt Boxing.
2. Seine Aktionen werden gespeichert.
3. Unser Keras-Student lernt diese Aktionen nachzumachen.
4. Danach spielt der Student selbst.
5. Der Teacher beschriftet schwierige Situationen des Students.
6. Das Modell wird erneut trainiert und evaluiert.
7. Das beste Modell wird gespeichert.

Finaler Trainingsstand:

- `337` abgeschlossene Trainingsrunden
- `20,202` gelabelte Episoden
- `8,966,962` Trainingsbeispiele

Bestes 10-Spiele-Ergebnis:

100, 100, 100, 100, 100, 100, 100, 100, 100, 100

Finale 5-Spiele-Evaluation:

98, 100, 100, 100, 98

Durchschnitt:

99.2

Beweisdateien

- Finales Modell: `models/mohi\_boxer\_best.h5`
- Reward-Kurve: `artifacts/training/training\_reward\_curve.png`
- Evaluation: `artifacts/evaluation/final\_5\_game\_summary.json`
- Videos: `artifacts/evaluation/videos/`
- Modellzusammenfassung: `artifacts/training/model\_summary.txt`

Hyperparameter

Finaler Abgabe-Agent

Parameter	Wert
---	---
Environment	`ALE/Boxing-v5`
Finales Modell	`models/mohi_boxer_best.h5`
Modellformat	Keras `.h5` plus `.keras` Backup
Finaler Trainingsansatz	Teacher-Student Distillation mit DAgger-ähnlichen Rescue-Runden
Aktion beim Spielen	Greedy `argmax(logits)`
Teacher-Familie	CleanRL PPO Atari Boxing Teacher
Optimizer des Students	Adam
Loss des Students	Sparse Categorical Crossentropy aus Logits
Learning Rate des Students	`1e-4` während Sticky-Rescue
Batch Size	`256` während Sticky-Rescue
Epochen pro Rescue-Runde	`3`
Validation Fraction	`0.05`
Shuffle Buffer	`8192`
Abgeschlossene Rescue-Runden	`337`
Teacher/Student-gelabelte Episoden	`20,202`
Teacher/Student-gelabelte Samples	`8,966,962`
Bestes 10-Spiele-Ergebnis	`100.0` Durchschnitt
Pflicht-5-Spiele-Ergebnis	`99.2` Durchschnitt

Environment-Vorverarbeitung

Parameter	Wert
---	---
Raw Observation	Atari RGB Frame `(210, 160, 3)`
Vorverarbeiteter Input	4 gestapelte Graustufenbilder
Modell-Input-Shape	`(4, 84, 84)`
Bildgröße	`84`
Frame Stack	`4`
Frame Skip	`4`
No-op Max	`30`
Full Action Space	`True`
Anzahl Aktionen	`18`
Sticky Action Probability	`0.25`
Terminal on Life Loss	`False`

Neuronales Netz

Komponente	Wert
---	---
Parameter	`1,093,858`
Input dtype	`uint8`

Input Scaling	`1 / 255`
CNN-Aufbau	IMPALA-ähnliches Residual CNN
Conv Sequence Channels	`16`, `32`, `32`
Residual Blocks	2 Residual Blocks pro Conv Sequence
Dense Layer	`256` ReLU-Neuronen
Output Layer	`18` Logits

Baseline DQN / DDQN

Das Projekt enthält zusätzlich den ursprünglichen DQN/DDQN-Code. Dieser war der erste Ansatz vor dem finalen Teacher-Student-Training.

Parameter	Wert
Algorithmus	DQN oder DDQN
Learning Rate	`1e-4`
Discount Factor Gamma	`0.99`
Batch Size	`32`
Replay Memory Size	`50,000`
Replay Type	standardmäßig Prioritized Replay
Epsilon Start	`1.0`
Epsilon Ende	`0.1`
Target Network Update	alle `1000` Trainingsschritte
Warmup Steps	`2000`
Train Frequency	alle `4` Environment-Schritte
Checkpoint Frequency	alle `25` Episoden

Evaluationsbericht

Umgebung

- Environment: `ALE/Boxing-v5`
- Gegner: eingebauter Atari-Boxing-Gegner
- Observation: 4 gestapelte Graustufenbilder mit `84x84` Pixeln
- Action Space: `Discrete(18)`
- Sticky Actions: `repeat\_action\_probability=0.25`

Pflicht-Evaluation Über 5 Spiele

Lokaler Befehl zur erneuten Evaluation:

```
python src/boxing.py evaluate --algorithm dqn --model models/mohi_boxer_best.h5 --episodes 5 --video  
--output-dir artifacts/evaluation_check
```

Abgabe-Modell:

```
models/mohi_boxer_best.h5
```

Ergebnis:

Spiel	Reward
---	---:
1	98
2	100
3	100
4	100
5	98

Durchschnittsreward:

```
99.2
```

Standardabweichung:

```
0.9798
```

Lokale Beweise:

- Zusammenfassung: `artifacts/evaluation/final\_5\_game\_summary.json`
- Videos: `artifacts/evaluation/videos/`

Beste 10-Spiele-Evaluation

Während des Trainings wurde ein perfektes 10-Spiele-Ergebnis erreicht:

- 10 Spiele
- alle 10 Rewards waren `100`
- Durchschnitt: `100.0`
- Standardabweichung: `0.0`

Lokale Beweise:

- `artifacts/training/best\_10\_game\_evaluation.json`
- `artifacts/training/training\_overview.json`

Reflexion

Was gut funktioniert hat

Am besten funktioniert hat der Wechsel von normalem DQN/DDQN zu Teacher-Student-Training.

Bei DQN/DDQN muss der Agent sehr viel selbst ausprobieren. Das ist bei Atari Boxing schwierig, weil gutes Verhalten nicht sofort offensichtlich ist. Der Agent muss lernen, wie weit er vom Gegner entfernt sein soll, wann er schlagen soll und wie er sich verteidigt.

Mit Teacher-Student-Training war das deutlich schneller. Ein starker Teacher-Agent hat gute Aktionen vorgemacht. Unser Keras-Modell konnte diese Aktionen nachlernen.

Auch die wiederholten Rescue-Runden haben geholfen. Dabei spielt der Student selbst, und der Teacher beschriftet dann die Situationen, in denen der Student landet. Dadurch lernt das Modell nicht nur perfekte Teacher-Situationen, sondern auch Situationen, die durch eigene Fehler entstehen.

Wichtig war außerdem das Training mit Sticky Actions. Bei Sticky Actions wird manchmal die vorherige Aktion wiederholt. Dadurch ist das Spiel realistischer und schwieriger. Frühere Modelle waren ohne Sticky Actions stark, aber mit Sticky Actions schwächer. Das finale Modell wurde deshalb robuster trainiert.

Was nicht so gut funktioniert hat

Reines DDQN war für die verfügbare Zeit zu langsam. Der Code enthält Replay Memory, Target Network und Epsilon-Greedy Exploration, aber Atari Boxing braucht sehr viele Spiele, bis ein Agent allein durch Ausprobieren gut wird.

Das GPU-Setup war auch schwierig. Auf dem Server gab es moderne RTX-GPUs, aber nicht jede TensorFlow-Version funktionierte direkt damit. Deshalb war CPU-Fallback wichtig. Für die Abgabe ist CPU-Evaluation am sichersten, weil sie zuverlässiger läuft.

Ein weiterer wichtiger Punkt: Das neueste Modell war nicht automatisch das beste Modell. Die letzte Runde hatte einen schlechteren Durchschnitt als ein früher gespeichertes Modell. Deshalb wurde für die Abgabe bewusst das beste gespeicherte Modell verwendet.

Warum diese Architektur sinnvoll ist

Der Agent bekommt Bilder als Input. Deshalb ist ein Convolutional Neural Network sinnvoll.

Ein CNN kann räumliche Informationen aus Bildern lernen, zum Beispiel:

- wo der eigene Boxer steht
- wo der Gegner steht
- ob ein Schlag möglich ist
- wie sich die Boxer bewegen

Das finale Netz ist nicht extrem groß. Es hat ungefähr `1.09 Millionen` Parameter. Das ist groß genug für Atari-Bilder, aber noch klein genug, um schnell zu trainieren und zu evaluieren.

Was ich nächstes Mal anders machen würde

Ich würde früher klären, wie genau das Turnier ablaufen wird. Dann könnte man gezielt gegen verschiedene Gegnertypen trainieren und nicht nur gegen den eingebauten Atari-Gegner.

Außerdem würde ich von Anfang an alle Experimente sauberer in eigenen Ordnern speichern. Dann wären Trainingskurven, Modelle, Videos und Konfigurationen noch einfacher zu vergleichen.

Als nächsten technischen Schritt würde ich gegen mehrere verschiedene Agenten trainieren. Das Ziel wäre, nicht nur gegen den Standard-Gegner gut zu sein, sondern auch gegen Mitschüler-Modelle im Turnier.

Fazit

Das finale Modell ist für die Abgabe geeignet, weil:

- es als Keras `.h5` Modell gespeichert ist
- es direkt über den README-Befehl geladen werden kann
- es eine Reward-Kurve gibt
- es eine 5-Spiele-Evaluation gegen den Standard-Gegner gibt
- es Videos von dieser Evaluation gibt
- die wichtigsten Entscheidungen dokumentiert sind

Quellen

- Gymnasium Boxing Dokumentation: <https://gymnasium.farama.org/v0.27.0/environments/atari/boxing/>
- CleanRL Boxing Teacher-Modellfamilie:
https://huggingface.co/cleanrl/Boxing-v5-cleanba_ppo_envpool_impala_atari_wrapper-seed2
- CleanRL Paper: <https://arxiv.org/abs/2111.08819>
- EnvPool Paper: <https://arxiv.org/abs/2206.10558>

Wichtig: Das finale abgegebene Modell ist nicht einfach das externe Teacher-Modell. Das finale Modell ist unser eigenes Keras-Student-Modell ``models/mohi_boxer_best.h5``. Es wurde mit Teacher/Student-gelabelten Trajektorien trainiert und danach über Evaluationen als bestes Modell ausgewählt.